

Experiment 6: Bag of Words (BoW) Model Implementation

Aim

To implement the Bag of Words (BoW) model using Python in Jupyter Notebook.

Theory

The Bag of Words (BoW) model is a fundamental technique in Natural Language Processing (NLP) used to convert text data into numerical form. It represents text as a collection of words without considering grammar or word order. Each unique word in the dataset is treated as a feature, and the frequency of each word is counted. This results in a vector representation for each document. BoW is simple and efficient but ignores context and semantics. It is widely used in text classification, sentiment analysis, and information retrieval tasks. Despite its limitations, it serves as a strong baseline model in NLP.

Algorithm / Steps

1. Collect the dataset (documents).
2. Convert text into lowercase.
3. Tokenize the text into words.
4. Create a vocabulary of unique words.
5. Count frequency of each word in each document.
6. Represent each document as a vector.
7. Display the Bag of Words matrix.

```
# Import required library
from sklearn.feature_extraction.text import CountVectorizer

# Dataset
documents = [
    "NLP is very interesting",
    "Machine learning is powerful",
    "NLP and machine learning are related",
    "Deep learning is part of AI",
    "AI is transforming the world",
    "Students are learning NLP",
    "Python is used for machine learning",
    "NLP applications are growing",
    "Data science uses machine learning",
    "AI and NLP are closely related"
]

# Initialize CountVectorizer
```

```

vectorizer = CountVectorizer()

# Fit and transform data
X = vectorizer.fit_transform(documents)

# Get vocabulary
features = vectorizer.get_feature_names_out()

# Convert into array
bow_matrix = X.toarray()

# Print results
print("Vocabulary:\n", features)
print("\nBag of Words Matrix:\n", bow_matrix)

```

Vocabulary:

['ai' 'and' 'applications' 'are' 'closely' 'data' 'deep' 'for' 'growing' 'interesting' 'is' 'learning' 'machine' 'nlp' 'of' 'part' 'powerful' 'python' 'related' 'science' 'students' 'the' 'transforming' 'used' 'uses' 'very' 'world']

Bag of Words Matrix:

```

[[0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 1 0 0 0 0 0 0 0 0]
 [0 1 0 1 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 1 0 0 0 0 0 0]
 [1 0 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0 0 0 0 0 0 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0]
 [0 0 0 0 0 0 0 1 0 0 1 1 1 0 0 0 0 1 0 0 0 0 0 1 0 0]
 [0 0 1 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0]
 [1 1 0 1 1 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0]]

```

Output

- Vocabulary: List of all unique words from the dataset.
- Bag of Words Matrix: Numerical representation of documents.
- Each row represents a document.
- Each column represents a word from the vocabulary.
- Values indicate the frequency of words in each document.

Experiment 7: TF-IDF Vectorization

Aim

To implement TF-IDF vectorization using Python in Jupyter Notebook.

Theory

TF-IDF (Term Frequency–Inverse Document Frequency) is a numerical technique used in Natural Language Processing to evaluate the importance of a word in a document relative to a collection of documents. Term Frequency (TF) measures how often a word appears in a document, while Inverse Document Frequency (IDF) reduces the importance of words that appear frequently across all documents. The TF-IDF value increases with the number of times a word appears in a document but decreases if the word is common in many documents. This helps in identifying important words while ignoring less meaningful ones. TF-IDF is widely used in text mining, information retrieval, and search engines.

Algorithm / Steps

1. Collect the dataset (documents).
2. Convert text into lowercase.
3. Tokenize the text into words.
4. Calculate Term Frequency (TF) for each word in a document.
5. Calculate Inverse Document Frequency (IDF) for all words.
6. Multiply TF and IDF to get TF-IDF score.
7. Represent each document as a TF-IDF vector.
8. Display the TF-IDF matrix.

```
# Import required library
from sklearn.feature_extraction.text import TfidfVectorizer

# Dataset (same as previous experiment)
documents = [
    "NLP is very interesting",
    "Machine learning is powerful",
    "NLP and machine learning are related",
    "Deep learning is part of AI",
    "AI is transforming the world",
    "Students are learning NLP",
    "Python is used for machine learning",
    "NLP applications are growing",
    "Data science uses machine learning",
    "AI and NLP are closely related"
]

# Initialize TF-IDF Vectorizer
```

```

vectorizer = TfidfVectorizer()

# Fit and transform the dataset
X = vectorizer.fit_transform(documents)

# Get feature names (vocabulary)
features = vectorizer.get_feature_names_out()

# Convert to array
tfidf_matrix = X.toarray()

# Print results
print("Vocabulary:\n", features)
print("\nTF-IDF Matrix:\n", tfidf_matrix)

```

Vocabulary:

```

['ai' 'and' 'applications' 'are' 'closely' 'data' 'deep' 'for'
'growing'
'interesting' 'is' 'learning' 'machine' 'nlp' 'of' 'part' 'powerful'
'python' 'related' 'science' 'students' 'the' 'transforming' 'used'
'uses' 'very' 'world']

```

TF-IDF Matrix:

```

[[0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.60799021 0.36103726 0.
  0. 0.36103726 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  0. 0.60799021 0. ]
[0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0.41193567 0.37239968
  0.45869682 0. 0. 0. 0.69370362 0.
  0. 0. 0. 0. 0. 0.
  0. 0. 0. ]
[0. 0.49405838 0. 0.38429473 0. 0.
  0. 0. 0. 0. 0. 0.31199527
  0.38429473 0.34511839 0. 0. 0. 0.
  0.49405838 0. 0. 0. 0. 0.
  0. 0. 0. ]
[0.36316487 0. 0. 0. 0. 0.
  0.48830236 0. 0. 0. 0.28996412 0.26213449
  0. 0. 0.48830236 0.48830236 0. 0.
  0. 0. 0. 0. 0. 0.
  0. 0. 0. ]
[0.37632442 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0.3004712 0.
  0. 0. 0. 0. 0. 0.
  0. 0. 0. 0.50599637 0.50599637 0.
  0. 0. 0.50599637]
[0. 0. 0. 0.45869682 0. 0.
  0. 0. 0. 0. 0. 0.37239968

```

```

0.          0.41193567 0.          0.          0.          0.
0.          0.          0.69370362 0.          0.          0.
0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.
0.          0.49519326 0.          0.          0.29405608 0.26583372
0.32743605 0.          0.          0.          0.          0.49519326
0.          0.          0.          0.          0.          0.49519326
0.          0.          0.          ]
[0.          0.          0.59870078 0.39587821 0.          0.
0.          0.          0.59870078 0.          0.          0.
0.          0.35552101 0.          0.          0.          0.
0.          0.          0.          0.          0.          0.
0.          0.          0.          ]
[0.          0.          0.          0.          0.          0.51809938
0.          0.          0.          0.          0.          0.27813037
0.34258224 0.          0.          0.          0.          0.
0.          0.51809938 0.          0.          0.          0.
0.51809938 0.          0.          ]
[0.38211434 0.43676109 0.          0.33972703 0.51378135 0.
0.          0.          0.          0.          0.          0.
0.          0.30509408 0.          0.          0.          0.
0.43676109 0.          0.          0.          0.          0.
0.          0.          0.          ]]

```

Output

- Vocabulary: List of all unique words in the dataset.
- TF-IDF Matrix: Numerical representation of documents based on importance of words.
- Each row represents a document.
- Each column represents a word.
- Values are TF-IDF scores (not counts like BoW).
- Higher values indicate more important words in a document.

Conclusion

TF-IDF vectorization provides a better representation of text data compared to the Bag of Words model by considering the importance of words across all documents. It reduces the weight of commonly occurring words and highlights significant terms. This makes it more effective for tasks like text classification and information retrieval. However, it still does not capture word order or semantic meaning. Overall, TF-IDF is a powerful and widely used technique in NLP.

Experiment 8: Sentiment Analysis using TextBlob

Aim

To perform sentiment analysis on text data using the TextBlob library in Python.

Theory

Sentiment Analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone of a text. It classifies text into categories such as positive, negative, or neutral. TextBlob is a Python library that provides a simple API for common NLP tasks, including sentiment analysis. It uses a rule-based approach along with a predefined lexicon to calculate sentiment polarity and subjectivity. Polarity ranges from -1 (negative) to +1 (positive), while subjectivity ranges from 0 (objective) to 1 (subjective). Sentiment analysis is widely used in social media monitoring, customer feedback analysis, and product reviews.

Algorithm / Steps

1. Create or collect a dataset of text sentences.
2. Import the TextBlob library.
3. Process each sentence using TextBlob.
4. Calculate sentiment polarity and subjectivity.
5. Classify sentiment as positive, negative, or neutral.
6. Display the results.

```
!pip install textblob
```

```
Defaulting to user installation because normal site-packages is not writeable
```

```
Collecting textblob
```

```
  Downloading textblob-0.19.0-py3-none-any.whl.metadata (4.4 kB)
```

```
Requirement already satisfied: nltk>=3.9 in c:\users\samee\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from textblob) (3.9.3)
```

```
Requirement already satisfied: click in c:\users\samee\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from nltk>=3.9->textblob) (8.3.1)
```

```
Requirement already satisfied: joblib in c:\users\samee\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\site-packages (from nltk>=3.9->textblob) (1.5.3)
```

```
Requirement already satisfied: regex>=2021.8.3 in c:\users\samee\
```

```

appdata\local\packages\
pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-
packages\python313\site-packages (from nltk>=3.9->textblob)
(2026.2.28)
Requirement already satisfied: tqdm in c:\users\samee\appdata\local\
packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\
localcache\local-packages\python313\site-packages (from nltk>=3.9-
>textblob) (4.67.3)
Requirement already satisfied: colorama in c:\users\samee\appdata\
local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\
localcache\local-packages\python313\site-packages (from click-
>nltk>=3.9->textblob) (0.4.6)
Downloading textblob-0.19.0-py3-none-any.whl (624 kB)
----- 0.0/624.3 kB ? eta -:--:--
----- 0.0/624.3 kB ? eta -:--:--
----- 0.0/624.3 kB ? eta -:--:--
----- 262.1/624.3 kB ? eta
-:--:--
----- 524.3/624.3 kB 1.4 MB/s
eta 0:00:01
----- 624.3/624.3 kB 1.1 MB/s
0:00:00
Installing collected packages: textblob
Successfully installed textblob-0.19.0

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: C:\Users\Samee\AppData\Local\Microsoft\
WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\
python.exe -m pip install --upgrade pip

from textblob import TextBlob

# Sample dataset
sentences = [
    "I love this product, it is amazing!",
    "This is the worst experience I have ever had.",
    "The movie was okay, not great but not bad.",
    "I am very happy with the service.",
    "The food was terrible and disappointing.",
    "It is just an average performance.",
    "Absolutely fantastic work!",
    "I am not satisfied with the quality.",
    "The experience was wonderful and exciting.",
    "Nothing special, it was normal."
]

# Perform sentiment analysis
for sentence in sentences:
    blob = TextBlob(sentence)

```

```
polarity = blob.sentiment.polarity
subjectivity = blob.sentiment.subjectivity
```

```
# Classify sentiment
```

```
if polarity > 0:
    sentiment = "Positive"
elif polarity < 0:
    sentiment = "Negative"
else:
    sentiment = "Neutral"
```

```
# Print results
```

```
print("Sentence:", sentence)
print("Polarity:", polarity)
print("Subjectivity:", subjectivity)
print("Sentiment:", sentiment)
print("-" * 50)
```

```
Sentence: I love this product, it is amazing!
Polarity: 0.625
Subjectivity: 0.75
Sentiment: Positive
```

```
-----
Sentence: This is the worst experience I have ever had.
Polarity: -1.0
Subjectivity: 1.0
Sentiment: Negative
```

```
-----
Sentence: The movie was okay, not great but not bad.
Polarity: 0.14999999999999997
Subjectivity: 0.6388888888888888
Sentiment: Positive
```

```
-----
Sentence: I am very happy with the service.
Polarity: 1.0
Subjectivity: 1.0
Sentiment: Positive
```

```
-----
Sentence: The food was terrible and disappointing.
Polarity: -0.8
Subjectivity: 0.85
Sentiment: Negative
```

```
-----
Sentence: It is just an average performance.
Polarity: -0.15
Subjectivity: 0.39999999999999997
Sentiment: Negative
```

```
-----
Sentence: Absolutely fantastic work!
Polarity: 0.5
```



```
Subjectivity: 0.9
Sentiment: Positive
-----
Sentence: I am not satisfied with the quality.
Polarity: -0.25
Subjectivity: 1.0
Sentiment: Negative
-----
Sentence: The experience was wonderful and exciting.
Polarity: 0.65
Subjectivity: 0.9
Sentiment: Positive
-----
Sentence: Nothing special, it was normal.
Polarity: 0.25357142857142856
Subjectivity: 0.6107142857142857
Sentiment: Positive
-----
```

Output

- Each sentence is analyzed for sentiment.
- Polarity:
 - Positive (> 0)
 - Negative (< 0)
 - Neutral ($= 0$)
- Subjectivity:
 - Measures how subjective or objective the sentence is.
- The model classifies each sentence into Positive, Negative, or Neutral.

Conclusion

Sentiment analysis using TextBlob is simple and effective for analyzing the emotional tone of text data. It provides polarity and subjectivity scores that help in understanding opinions. The model can classify text into positive, negative, or neutral categories efficiently. However, it may not perform well on complex sentences, sarcasm, or context-dependent meanings. Overall, it is a useful tool for basic sentiment analysis tasks.

Experiment 9: Spam Detection using Naive Bayes Classifier

Aim

To implement spam detection using the Naive Bayes classifier in Python.

Theory

Spam detection is a text classification task in Natural Language Processing (NLP) used to identify whether a message is spam or not. The Naive Bayes classifier is a probabilistic machine learning algorithm based on Bayes' Theorem. It assumes that features (words) are independent of each other, which simplifies computation. In text classification, it calculates the probability of a message being spam or ham (not spam) based on word frequencies. It works efficiently with high-dimensional data and is widely used for email filtering and message classification. Despite its simplicity, Naive Bayes often performs well in real-world NLP problems.

Algorithm / Steps

1. Create or collect a labeled dataset (spam / ham messages).
2. Convert text data into numerical form using CountVectorizer or TF-IDF.
3. Split the dataset into training and testing sets.
4. Train the Naive Bayes classifier on training data.
5. Predict the labels for test data.
6. Evaluate the model using accuracy.
7. Display the results.

```
# Import required libraries
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Sample dataset
messages = [
    "Win a free iPhone now",
    "Congratulations! You have won a lottery",
    "Call me when you are free",
    "Let's meet for lunch tomorrow",
    "Exclusive offer just for you",
    "Important update about your account",
    "You have won a free ticket",
    "Can we discuss the project today?",
    "Limited time offer, claim now",
    "Are you coming to the class?"
```

```

]

# Labels: spam = 1, ham = 0
labels = [1, 1, 0, 0, 1, 0, 1, 0, 1, 0]

# Convert text into numerical features
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(messages)

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, labels,
test_size=0.3, random_state=42)

# Train Naive Bayes classifier
model = MultinomialNB()
model.fit(X_train, y_train)

MultinomialNB()

# Predict on test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Predictions:", y_pred)
print("Actual:", y_test)

Accuracy: 0.6666666666666666
Predictions: [1 1 1]
Actual: [1, 1, 0]

# Test with a new message
new_message = ["Free entry in a contest, win prizes now"]
new_X = vectorizer.transform(new_message)

prediction = model.predict(new_X)

print("Message:", new_message[0])
print("Prediction:", "Spam" if prediction[0] == 1 else "Ham")

Message: Free entry in a contest, win prizes now
Prediction: Spam

```

Output

- The dataset is converted into numerical features using CountVectorizer.
- The Naive Bayes model is trained on training data.
- Predictions are made on test data.
- Accuracy shows how well the model performs.

- New messages can also be classified as Spam or Ham.

Experiment 10: Simple Rule-Based NLP Chatbot

Aim

To implement a simple rule-based chatbot using Python in Jupyter Notebook.

Theory

A rule-based chatbot is a basic conversational system that responds to user input based on predefined rules and patterns. It does not use machine learning but relies on keyword matching and conditional statements. When a user enters a query, the chatbot searches for specific keywords and returns a corresponding response. These chatbots are simple, fast, and easy to implement but lack the ability to understand context or learn from interactions. They are commonly used in FAQs, customer support, and basic automation tasks. Despite their limitations, they are useful for understanding the fundamentals of conversational AI.

Algorithm / Steps

1. Define a set of predefined rules (keywords and responses).
2. Take user input from the console.
3. Convert input to lowercase for uniform processing.
4. Check for keywords in the input.
5. Match the input with predefined rules.
6. Return the corresponding response.
7. Repeat until the user exits the chatbot.

```
# Simple Rule-Based Chatbot

def chatbot_response(user_input):
    user_input = user_input.lower()

    # Rule-based responses
    if "hello" in user_input or "hi" in user_input:
        return "Hello! How can I help you?"

    elif "how are you" in user_input:
        return "I'm just a chatbot, but I'm doing great!"

    elif "your name" in user_input:
        return "I am a simple NLP chatbot."

    elif "help" in user_input:
        return "I can answer basic questions. Try asking something!"

    elif "bye" in user_input or "exit" in user_input:
        return "Goodbye! Have a nice day."
```

```

        else:
            return "Sorry, I didn't understand that."
# Run chatbot
print("Chatbot is running! Type 'exit' to stop.\n")
while True:
    user_input = input("You: ")

    response = chatbot_response(user_input)
    print("Bot:", response)

    if "exit" in user_input.lower():
        break
Chatbot is running! Type 'exit' to stop.
Bot: Hello! How can I help you?
Bot: I can answer basic questions. Try asking something!
Bot: I'm just a chatbot, but I'm doing great!
Bot: Goodbye! Have a nice day.
Bot: Goodbye! Have a nice day.

```

Output

- The chatbot takes user input from the console.
- It matches keywords with predefined rules.
- Based on the input, it generates appropriate responses.
- The chatbot continues conversation until the user types 'exit'.

Conclusion

The rule-based chatbot successfully responds to user queries using predefined rules. It demonstrates the basic working of conversational systems without using machine learning. While it is simple and efficient, it cannot handle complex queries or understand context. This experiment helps in understanding the foundation of chatbot development. More advanced chatbots use machine learning and NLP techniques for better interaction.